



INTERVIEW PREPARATION CODES

Data Structures & Algorithms

By Career Credentials



DSA

Stack Data Structure:

1. Implement a Stack:

Create a Any coding language class to implement a stack data structure with methods like **push**, **pop**, **peek**, and **is_empty**.
Use a list to simulate a stack and implement the methods accordingly.

2. Check Balanced Parentheses:

Write a Any coding language function to check if a given string containing parentheses is balanced (i.e., every opening parenthesis has a closing one).
Use a stack to keep track of open parentheses and ensure they are closed properly.

3. Evaluate Postfix Expression:

Write a Any coding language function to evaluate a postfix expression(e.g., "23*5+") using a stack.
Use a stack to process the postfix expression and calculate the result.

4. Implement a Min Stack:

Create a Any coding language stack that supports the **push**, **pop**, **top**, and **get_min** operations in $O(1)$ time.
Use an additional stack to keep track of the minimum element at any given time.

5. Reverse a Stack:

Write a Any coding language function to reverse the order of elements in a stack.
Use an auxiliary stack to reverse the elements from the original stack.

6. Implement Two Stacks in One Array:

Create a Any coding language class that implements two stacks in a single array efficiently.
Divide the array into two halves and manage two stacks independently.

7. Sort a Stack:

Write a Any coding language function to sort a stack using only one additional stack.
Use an auxiliary stack to perform the sorting operation.



8. Implement a Max Stack:

Create a Any coding language stack that supports the **push**, **pop**, **top**, and **get_max** operations in $O(1)$ time.

Use an additional stack to keep track of the maximum element at any given time.

Queue Data Structure:

9. Implement a Queue:

Create a Any coding language class to implement a queue data structure with methods like **enqueue**, **dequeue**, **peek**, and **is_empty**.

Use a list to simulate a queue and implement the methods accordingly.

10. Implement a Circular Queue:

Create a Any coding language class to implement a circular queue data structure with methods like **enqueue**, **dequeue**, and checks for full and empty states.

Use an array and maintain front and rear pointers.

11. Implement a Queue using Stacks:

Create a Any coding language queue using two stacks and implement the **enqueue** and **dequeue** operations.

Use two stacks to simulate the queue and perform enqueue and dequeue operations.

12. Implement a Double-Ended Queue (Deque):

Create a Any coding language class to implement a deque data structure with methods like **append**, **appendleft**, **pop**, and **popleft**.

Use a doubly-linked list to implement the deque.

13. Implement a Priority Queue:

Create a Any coding language priority queue data structure that supports **insert** and **get_highest_priority** operations.

Use a heap data structure to implement the priority queue.

14. Reorder a Queue:

Write a Any coding language function to reorder a given queue such that elements with even values appear before elements with odd values, without using any extra data structures.

Use two queues to separate even and odd elements.

15. Implement a Blocking Queue:

Create a Any coding language blocking queue that supports **put** and **get** operations, which block when the queue is empty or full.

Use threading or multiprocessing to implement the blocking behavior.



16. Implement a Queue using Two Stacks:

Create a Any coding language queue using two stacks and implement the **enqueue** and **dequeue** operations.

Use two stacks to simulate the queue and perform enqueue and dequeue operations.

17. Generate Binary Numbers using a Queue:

Write a Any coding language function to generate binary numbers from 1 to n using a queue.

Use a queue to implement a breadth-first search (BFS) approach to generate binary numbers.

18. Implement a Sliding Window Maximum:

Write a Any coding language function to find the maximum element in a sliding window of fixed size in an array.

Use a deque to maintain the maximum values within the sliding

Linked List Basics:

1. Implement a Singly Linked List:

Create a Any coding language class to represent a singly linked list with methods like **insert_at_head**, **insert_at_tail**, **delete**, and **search**.

Implement the methods to manipulate the linked list accordingly.

2. Find the Length of a Linked List:

Write a function to find the length of a singly linked list.

Traverse the linked list and count the number of nodes.

3. Reverse a Linked List:

Write a function to reverse a singly linked list.

Use iterative or recursive methods to reverse the pointers of the nodes.

4. Detect a Cycle in a Linked List:

Write a function to detect if a singly linked list has a cycle.

Use Floyd's cycle-finding algorithm (two-pointer technique) to detect cycles in the linked list.

5. Find the Middle of a Linked List:

Write a function to find the middle node of a singly linked list.

Use the slow and fast pointer approach to find the middle node.



Advanced Linked List Problems:

6. Remove Nth Node from End of List:

Write a function to remove the nth node from the end of a singly linked list.
Use two pointers to find the appropriate node and remove it.

7. Merge Two Sorted Linked Lists:

Write a function to merge two sorted singly linked lists into a single sorted list.
Compare nodes of both lists and merge them into a new linked list.

8. Intersection Point in Y Shaped Linked Lists:

Write a function to find the node where two linked lists intersect.
Traverse both lists, find their lengths, align them, and then find the intersection node.

9. Detect a Cycle's Starting Point:

Write a function to find the starting node of a cycle in a linked list.
Use Floyd's cycle-finding algorithm and then move one pointer to the start and iterate until they meet.

10. Palindrome Linked List:

Write a function to check if a linked list is a palindrome.
Reverse the second half of the linked list and compare it with the first half.

11. Flatten a Multilevel Doubly Linked List:

Write a function to flatten a multilevel doubly linked list into a singly linked list.
Flatten the list while maintaining the next and child pointers.

Linked List Operations:

12. Add Two Numbers Represented by Linked Lists:

Write a function to add two numbers represented by linked lists.
Traverse both lists simultaneously, calculate the sum, and create a new linked list for the result.

13. Intersection of Two Linked Lists:

Write a function to find the node where two linked lists intersect.
Find the lengths of both lists, align them, and then traverse until the intersection point is found.



14. Remove Duplicates from Sorted List:

Write a function to remove duplicates from a sorted linked list.
Traverse the list and remove duplicates by adjusting the pointers.

15. Rotate a Linked List:

Write a function to rotate a linked list by k places.
Find the length, calculate the new head, and adjust pointers accordingly.

Special Types of Linked Lists:

16. Clone a Linked List with Next and Random Pointer:

Write a function to clone a linked list with next and random pointers.
Create new nodes and map the relationships between original and newnodes.

17. LRU Cache Implementation using Linked List:

Implement an LRU (Least Recently Used) cache using a combination of a linked list and a dictionary.
Maintain the order of items based on usage and update the cache accordingly.

18. Flatten a Binary Tree to Linked List:

Write a function to flatten a binary tree into a singly linked list in-place.
Use a recursive approach to flatten the binary tree.

Variations of Linked Lists:

19. Add Two Numbers Represented by Reverse Linked Lists:

Write a function to add two numbers represented by reverse linked lists.
Traverse both lists simultaneously, calculate the sum, and create anew linked list for the result.

20. Rearrange a Linked List in Zigzag Fashion:

Write a function to rearrange a linked list in a zigzag fashion.
Traverse the list and reorder nodes based on the given pattern.

21. Rotate a Linked List:

Write a function to rotate a linked list by k nodes from the end.
Use the length of the list to find the new head after rotation.

22. Partition a Linked List around a Given Value:

Write a function to partition a linked list around a given value x.
Maintain two separate linked lists for nodes smaller and greater than x, then concatenate them.



23. Convert Binary Number in a Linked List to Integer:

Write a function to convert a binary number represented by a linked list to an integer.

Traverse the list, convert the binary number, and return the integer value.

24. Add One to a Linked List Representing a Number:

Write a function to add one to a number represented by a linked list.

Reverse the list, add one, and handle carry appropriately.

25. Copy Random Pointer Linked List:

Write a function to copy a linked list with next and random pointers in $O(n)$ time.

Create a new node for each original node and use a dictionary to map original nodes to new nodes.

Binary Tree Basics:

1. Implement a Binary Tree:

Create a Any coding language class to represent a binary tree with methods to insert and traverse nodes.

Implement the methods to manipulate and traverse the binary tree.

2. Find the Height of a Binary Tree:

Write a function to find the height (depth) of a binary tree.

Use a recursive approach to traverse the tree and calculate the height.

3. Check if a Binary Tree is Balanced:

Write a function to check if a binary tree is balanced (the heights of the two subtrees of every node never differ by more than one).

Use a recursive approach to check the balance of each node.

4. Serialize and Deserialize a Binary Tree:

Implement functions to serialize and deserialize a binary tree.

Use a depth-first search (DFS) or breadth-first search (BFS) approach to serialize and deserialize.

5. Find the Lowest Common Ancestor in a Binary Tree:

Write a function to find the lowest common ancestor of two nodes in a binary tree.

Use a recursive approach to search for the nodes and their common ancestor.



Binary Search Tree (BST):

6. Implement a Binary Search Tree (BST):

Create a Any coding language class to represent a binary search tree with methods to insert, search, and delete nodes.

Implement the methods to manipulate and search in the BST.

7. Validate a Binary Search Tree (BST):

Write a function to determine if a binary tree is a valid binary search tree.

Use a recursive approach to validate the properties of a BST.

8. Find the Inorder Successor in a BST:

Write a function to find the inorder successor of a given node in a BST.

Use a recursive approach to find the successor.

9. Convert a Sorted Array to a Balanced BST:

Write a function to convert a sorted array into a balanced binary search tree.

Use a recursive approach to build the tree from the sorted array.

10. Recover a BST with Two Swapped Nodes:

Write a function to recover a binary search tree where two nodes are swapped.

Use an inorder traversal to detect the swapped nodes and correct them.

Tree Traversals:

11. Preorder Traversal:

Write a function to perform a preorder traversal of a binary tree.

Use a recursive approach to visit nodes in the order: root, left, right.

12. Inorder Traversal:

Write a function to perform an inorder traversal of a binary tree.

Use a recursive approach to visit nodes in the order: left, root, right.

13. Postorder Traversal:

Write a function to perform a postorder traversal of a binary tree.

Use a recursive approach to visit nodes in the order: left, right, root.

14. Level-Order Traversal:

Write a function to perform a level-order traversal of a binary tree.

Use a queue to visit nodes level by level.



Advanced Tree Problems:

15. Find the Diameter of a Binary Tree:

Write a function to find the diameter (longest path) of a binary tree.
Use a recursive approach to calculate the diameter.

16. Count the Number of Unival Subtrees:

Write a function to count the number of unival (same value) subtrees in a binary tree.
Use a recursive approach to check for unival subtrees.

17. Check if a Binary Tree is Symmetric:

Write a function to check if a binary tree is symmetric (a mirror image of itself).
Use a recursive approach to check for symmetry.

18. Flatten a Binary Tree into a Linked List:

Write a function to flatten a binary tree into a singly linked list in-place.
Use a recursive approach to flatten the binary tree.

19. Find All Paths in a Binary Tree:

Write a function to find all paths from the root to leaf nodes in a binary tree.
Use a recursive approach to collect paths.

Special Types of Trees:

20. Convert Sorted Linked List to Binary Search Tree:

Write a function to convert a sorted linked list into a balanced binary search tree.
Use a recursive approach to build the tree from the linked list.

21. Check if a Binary Tree is Subtree of Another Binary Tree:

Write a function to check if one binary tree is a subtree of another binary tree.
Use a recursive approach to search for the subtree.

22. Serialize and Deserialize N-ary Tree:

Implement functions to serialize and deserialize an N-ary tree.
Use a depth-first search (DFS) approach for serialization and deserialization.

Variations of Tree Problems:

23. Find the kth Smallest Element in a BST:

Write a function to find the kth smallest element in a binary search tree.
Use an inorder traversal with a counter to find the kth smallest element.



24. Convert Binary Tree to Doubly Linked List:

Write a function to convert a binary tree into a doubly linked list in-place.

Use a recursive approach to convert the tree.

25. Count the Number of Nodes in a Complete Binary Tree:

Write a function to count the number of nodes in a complete binary tree.

Use a binary search approach to count nodes efficiently.

Miscellaneous Tree Problems:

26. Implement an Expression Tree:

Create a Any coding language class to represent an expression tree and evaluate expressions.

Use a stack to build and evaluate expressions.

27. Find the Closest Leaf in a Binary Tree:

Write a function to find the closest leaf node to a given target node in a binary tree.

Use a depth-first search to find the closest leaf.

28. Sum of All Left Leaves in a Binary Tree:

Write a function to find the sum of all left leaves in a binary tree.

Use a depth-first search to identify left leaves and calculate the sum.

29. Construct Binary Tree from Preorder and Inorder Traversal:

Write a function to construct a binary tree from its preorder and inorder traversals.

Use recursive methods to construct the tree.

30. Path Sum III:

Write a function to count the total number of paths that sum to a given value in a binary tree.

Use a recursive approach to explore paths and count valid sum.

Graph Basics:

1. Implement an Undirected Graph:

Create a Any coding language class to represent an undirected graph with methods to add vertices and edges.

Implement the methods to manipulate the graph.



2. Implement a Directed Graph:

Create a Any coding language class to represent a directed graph with methods to add vertices and directed edges.

Implement the methods to manipulate the directed graph.

3. Depth-First Search (DFS):

Write a function to perform a depth-first search on a graph.

Use recursion to traverse the graph and visit nodes in a specific order.

4. Breadth-First Search (BFS):

Write a function to perform a breadth-first search on a graph.

Use a queue to traverse the graph level by level.

5. Topological Sort:

Write a function to perform a topological sort on a directed acyclic graph (DAG).

Use DFS or BFS to obtain a topological order of the nodes.

Shortest Paths:

6. Dijkstra's Algorithm:

Implement Dijkstra's algorithm to find the shortest path in a weighted graph with non-negative edge weights.

Use a priority queue or min-heap to find the shortest path.

7. Bellman-Ford Algorithm:

Implement the Bellman-Ford algorithm to find the shortest path in a weighted graph with negative edge weights and detect negative cycles.

Use dynamic programming and relaxation steps.

8. Floyd-Warshall Algorithm:

Implement the Floyd-Warshall algorithm to find all shortest paths in a graph.

Use dynamic programming to calculate the shortest paths between all pairs of nodes.

9. Single-Source Shortest Path in a DAG:

Write a function to find the shortest paths from a single source node in a directed acyclic graph (DAG).

Use topological sorting and dynamic programming.



Graph Connectivity:

10. Connected Components:

Write a function to find and count the connected components in an undirected graph.

Use DFS or BFS to explore the graph.

11. Minimum Spanning Tree (Prim's Algorithm):

Implement Prim's algorithm to find the minimum spanning tree of an undirected, weighted graph.

Use a priority queue or min-heap to grow the minimum spanning tree.

12. Minimum Spanning Tree (Kruskal's Algorithm):

Implement Kruskal's algorithm to find the minimum spanning tree of an undirected, weighted graph.

Use a disjoint-set (union-find) data structure to grow the minimum spanning tree.

13. Strongly Connected Components (Kosaraju's Algorithm):

Write a function to find and count strongly connected components in a directed graph.

Use two DFS traversals to determine strongly connected components.

Advanced Graph Problems:

14. Word Ladder:

Write a function to find the length of the shortest transformation sequence from the start word to the end word by changing one character at a time.

Use BFS to explore word transformations.

15. Minimum Cut in a Graph:

Implement the Karger's algorithm to find the minimum cut in an undirected graph.

Use random contractions to identify the minimum cut.

16. Max Flow in a Network (Ford-Fulkerson Algorithm):

Implement the Ford-Fulkerson algorithm to find the maximum flow in a network.

Use augmenting paths to increase the flow.

17. Bipartite Graph Check:

Write a function to check if a graph is bipartite (can be divided into two sets such that there are no edges within a set).

Use BFS or DFS to assign colors to nodes and check for conflicts.



Graph Applications:

18. Shortest Path in a Grid:

Write a function to find the shortest path in a grid with obstacles.
Use BFS or Dijkstra's algorithm to navigate the grid.

19. Minimum Height Trees:

Write a function to find the minimum height trees in a tree-like undirected graph.
Use a topological sorting approach.

20. Redundant Connection in a Directed Graph:

Write a function to find a redundant connection in a directed graph.
Use DFS or union-find to detect cycles and find the redundant edge.

Variations of Graph Problems:

21. Largest Component Size by Common Factor:

Write a function to find the size of the largest component in an undirected graph where two numbers are connected if they share a common factor.
Use graph traversal to find the connected components.

22. Course Schedule (Directed Graph):

Write a function to check if it's possible to finish all courses in a course schedule based on prerequisites.
Use a directed graph and topological sorting.

23. Find the Celebrity:

Write a function to find the celebrity in a party, where everyone knows the celebrity, but the celebrity doesn't know anyone.
Use a graph-based elimination approach.

Miscellaneous Graph Problems:

24. Valid Tree:

Write a function to check if a given graph is a valid tree.
Use graph properties to check for cycles and connectivity.

25. Detect Cycles in a Directed Graph:

Write a function to detect cycles in a directed graph.
Use DFS to identify back edges in the graph.

26. Maximal Network Rank:

Write a function to find the maximal network rank in a graph.
Use degree calculations and adjacency lists.



27. Minimum Knight Moves:

Write a function to find the minimum number of moves required for a knight to move from one cell to another on a chessboard.
Use BFS to explore possible moves.

28. Number of Operations to Make Network Connected:

Write a function to find the minimum number of operations needed to make a network connected.
Use disjoint-set (union-find) data structure to determine connected components.

29. Redundant Connection (Undirected Graph):

Write a function to find a redundant connection in an undirected graph.
Use union-find to detect cycles and find the redundant edge.

30. Evaluate Division:

Write a function to evaluate division equations given as pairs of strings and their values.
Use a graph-based approach to perform division operations.